Unit I

An algorithm is a set of steps of operations to solve a problem performing calculation, data processing, and automated reasoning tasks. An algorithm is an efficient method that can be expressed within finite amount of time and space.

An algorithm is the best way to represent the solution of a particular problem in a very simple and efficient way. If we have an algorithm for a specific problem, then we can implement it in any programming language, meaning that the algorithm is independent from any programming languages.

Algorithm Design

The important aspects of algorithm design include creating an efficient algorithm to solve a problem in an efficient way using minimum time and space.

To solve a problem, different approaches can be followed. Some of them can be efficient with respect to time consumption, whereas other approaches may be memory efficient. However, one has to keep in mind that both time consumption and memory usage cannot be optimized simultaneously. If we require an algorithm to run in lesser time, we have to invest in more memory and if we require an algorithm to run with lesser memory, we need to have more time.

Problem Development Steps

The following steps are involved in solving computational problems.

- Problem definition
- Development of a model
- Specification of an Algorithm
- Designing an Algorithm
- Checking the correctness of an Algorithm
- Analysis of an Algorithm
- Implementation of an Algorithm
- Program testing
- Documentation

Characteristics of Algorithms

The main characteristics of algorithms are as follows –

- Algorithms must have a unique name
- Algorithms should have explicitly defined set of inputs and outputs
- Algorithms are well-ordered with unambiguous operations
- Algorithms halt in a finite amount of time. Algorithms should not run for infinity, i.e., an algorithm must end at some point

Pseudocode

Pseudocode gives a high-level description of an algorithm without the ambiguity associated with plain text but also without the need to know the syntax of a particular programming language.

The running time can be estimated in a more general manner by using Pseudocode to represent the algorithm as a set of fundamental operations which can then be counted.

Difference between Algorithm and Pseudocode

An algorithm is a formal definition with some specific characteristics that describes a process, which could be executed by a Turing-complete computer machine to perform a specific task. Generally, the word "algorithm" can be used to describe any high level task in computer science. On the other hand, pseudocode is an informal and (often rudimentary) human readable description of an algorithm leaving many granular details of it. Writing a pseudocode has no restriction of styles and its only objective is to describe the high level steps of algorithm in a much realistic manner in natural language.

Performance Analysis

we will discuss the need for analysis of algorithms and how to choose a better algorithm for a particular problem as one computational problem can be solved by different algorithms.

By considering an algorithm for a specific problem, we can begin to develop pattern recognition so that similar types of problems can be solved by the help of this algorithm.

Algorithms are often quite different from one another, though the objective of these algorithms is the same. For example, we know that a set of numbers can be sorted using different algorithms. Number of comparisons performed by one algorithm may vary with others for the same input. Hence, time complexity of those algorithms may differ. At the same time, we need to calculate the memory space required by each algorithm.

Analysis of algorithm is the process of analyzing the problem-solving capability of the algorithm in terms of the time and size required (the size of memory for storage while implementation). However, the main concern of analysis of algorithms is the required time or performance.

Generally, we perform the following types of analysis –

- Worst-case – The maximum number of steps taken on any instance of size a.
- Best-case – The minimum number of steps taken on any instance of size a.
- Average case – An average number of steps taken on any instance of size a.
- Amortized – A sequence of operations applied to the input of size a averaged over time.

To solve a problem, we need to consider time as well as space complexity as the program may run on a system where memory is limited but adequate space is available or may be vice-versa. In this context, if we compare bubble sort and merge sort. Bubble sort does not require additional memory, but merge sort requires additional space. Though time complexity of bubble sort is higher compared to merge sort, we may need to apply bubble sort if the program needs to run in an environment, where memory is very limited.

Time Complexity

It's a function describing the amount of time required to run an algorithm in terms of the size of the input. "Time" can mean the number of memory accesses performed, the number of comparisons between integers, the number of times some inner loop is executed, or some other natural unit related to the amount of real time the algorithm will take.

Space Complexity

It's a function describing the amount of memory an algorithm takes in terms of the size of input to the algorithm. We often speak of "extra" memory needed, not counting the memory needed to store the input itself. Again, we use natural (but fixed-length) units to measure this.

Space complexity is sometimes ignored because the space used is minimal and/or obvious, however sometimes it becomes as important an issue as time.

Asymptotic Notations

Execution time of an algorithm depends on the instruction set, processor speed, disk I/O speed, etc. Hence, we estimate the efficiency of an algorithm asymptotically.

Time function of an algorithm is represented by T(n), where n is the input size.

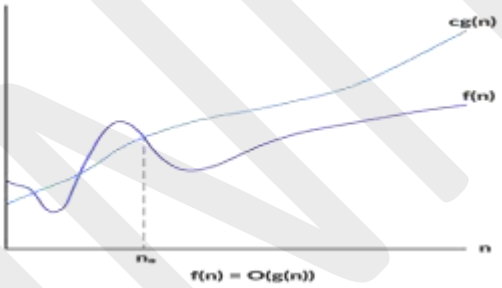
Different types of asymptotic notations are used to represent the complexity of an algorithm. Following asymptotic notations are used to calculate the running time complexity of an algorithm.

- O – Big Oh
- Ω – Big omega
- θ – Big theta
- o – Little Oh
- ω – Little omega

Big Oh Notation

Big-Oh (O) notation gives an upper bound for a function f(n) to within a constant factor. We write f(n) = O(g(n)), If there are positive constants n0 and c such that, to the right of n0 the f(n) always lies on or below c*g(n).

O(g(n)) = { f(n) : There exist positive constant c and n0 such that $0 \le f(n) \le c\, g(n)$, for all $n \le n0$}



Little o Notations

There are some other notations present except the Big-Oh, Big-Omega and Big-Theta notations. The little o notation is one of them.

Little o notation is used to describe an upper bound that cannot be tight. In other words, loose upper bound of f(n).

Let f(n) and g(n) are the functions that map positive real numbers. We can say that the function f(n) is o(g(n)) if for any real positive constant c, there exists an integer constant $n0 \le 1$ such that $f(n) > 0$.

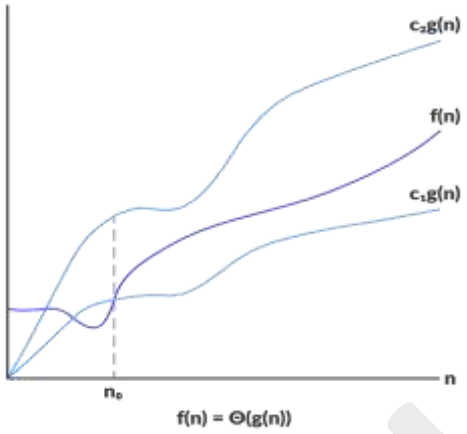
Omega Notation (Ω-notation)

Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.

Ω(g(n)) = { f(n): there exist positive constants c and n0
such that $0 \le cg(n) \le f(n)$ for all $n \ge n0$ }

The above expression can be described as a function f(n) belongs to the set Ω(g(n)) if there exists a positive constant c such that it lies above cg(n), for sufficiently large n.

For any value of n, the minimum time required by the algorithm is given by Omega Ω(g(n))



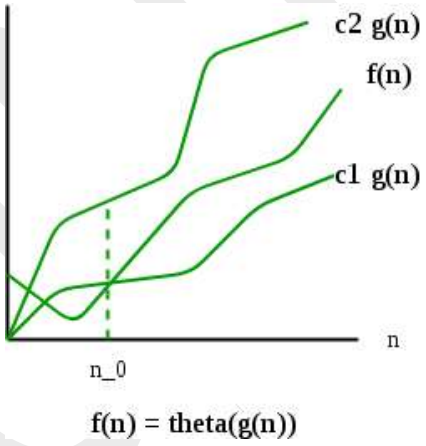
Theta Notation (Θ-notation)

For a function g(n), Θ(g(n)) is given by the relation:

Θ(g(n)) = { f(n): there exist positive constants c1, c2 and n0
such that 0 ≤ c1g(n) ≤ f(n) ≤ c2g(n) for all n ≥ n0 }

The above expression can be described as a function f(n) belongs to the set Θ(g(n)) if there exist positive constants c1 and c2 such that it can be sandwiched between c1g(n) and c2g(n), for sufficiently large n.

If a function f(n) lies anywhere in between c1g(n) and c2g(n) for all n ≥ n0, then f(n) is said to be asymptotically tight bound



Master Theorem

The master method is a formula for solving recurrence relations of the form:

T(n) = aT(n/b) + f(n),

where,

n = size of input

a = number of subproblems in the recursion

n/b = size of each subproblem. All subproblems are assumed to have the same size.

f(n) = cost of the work done outside the recursive call, which includes the cost of dividing the problem and cost of merging the solutions

Here, a ≥ 1 and b > 1 are constants, and f(n) is an asymptotically positive function

An asymptotically positive function means that for a sufficiently large value of n, we have

f(n) > 0.

The master theorem is used in calculating the time complexity of recurrence relations (divide and conquer algorithms) in a simple and quick way.

$T(n) = aT(n/b) + f(n)$,

where,

n = size of input

a = number of subproblems in the recursion

n/b = size of each subproblem. All subproblems are assumed to have the same size.

f(n) = cost of the work done outside the recursive call, which includes the cost of dividing the problem and cost of merging the solutions

Here, $a \geq 1$ and $b > 1$ are constants, and f(n) is an asymptotically positive function.

If $a \geq 1$ and $b > 1$ are constants and f(n) is an asymptotically positive function, then the time complexity of a recursive relation is given by

$T(n) = aT(n/b) + f(n)$

where, T(n) has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} * \log n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$, then $T(n) = \Theta(f(n))$.

$\epsilon > 0$ is a constant.

Each of the above conditions can be interpreted as:

1. If the cost of solving the sub-problems at each level increases by a certain factor, the value of f(n) will become polynomially smaller than $n^{\log_b a}$. Thus, the time complexity is oppressed by the cost of the last level ie. $n^{\log_b a}$
2. If the cost of solving the sub-problem at each level is nearly equal, then the value of f(n) will be $n^{\log_b a}$. Thus, the time complexity will be f(n) times the total number of levels ie. $n^{\log_b a} * \log n$
3. If the cost of solving the subproblems at each level decreases by a certain factor, the value of f(n) will become polynomially larger than $n^{\log_b a}$. Thus, the time complexity is oppressed by the cost of f(n).

Solved Example of Master Theorem

$T(n) = 3T(n/2) + n^2$

Here,

$a = 3$

$n/b = n/2$

$f(n) = n^2$

$\log_b a = \log_2 3 \approx 1.58 < 2$

ie. $f(n) < n^{\log_b a + \epsilon}$, where, ϵ is a constant.

Case 3 implies here.

Thus, $T(n) = f(n) = \Theta(n^2)$

Master Theorem Limitations
The master theorem cannot be used if:

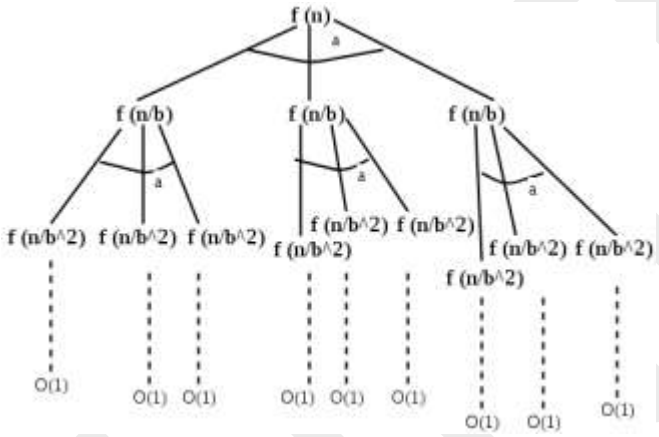
- T(n) is not monotone. eg. T(n) = sin n
- f(n) is not a polynomial. eg. f(n) = 2n
- a is not a constant. eg. a = 2n
- a < 1

T(n) = aT(n/b) + f(n) where a >= 1 and b > 1
There are following three cases:
1. If f(n) = Θ(nc) where c < Logba then T(n) = Θ(nLogba)

2. If f(n) = Θ(nc) where c = Logba then T(n) = Θ(ncLog n)

3.If f(n) = Θ(nc) where c > Logba then T(n) = Θ(f(n))

Master method is mainly derived from recurrence tree method. If we draw recurrence tree of T(n) = aT(n/b) + f(n), we can see that the work done at root is f(n) and work done at all leaves is Θ(nc) where c is Logba. And the height of recurrence tree is $Log_b n$



In recurrence tree method, we calculate total work done. If the work done at leaves is polynomially more, then leaves are the dominant part, and our result becomes the work done at leaves (Case 1). If work done at leaves and root is asymptotically same, then our result becomes height multiplied by work done at any level (Case 2). If work done at root is asymptotically more, then our result becomes work done at root (Case 3).

Examples :
1. Solve the following recurrence relation using Master's theorem-
T(n) = 3T(n/2) + n2
Sol:
We compare the given recurrence relation with T(n) = aT(n/b) + θ ($nklog_p n$).

Then, we have-a = 3,b = 2,k = 2,p = 0
Now, a = 3 and bk = 22 = 4.
Clearly, a < bk.
So, we follow case-03.
Since p = 0, so we have-
T(n) = θ (n k log_p n)

$T(n) = \theta (n2log0n)$
Thus,
$T(n) = \theta (n2)$

2:Solve the following recurrence relation using Master's theorem-

$T(n) = 2T(n/2) + nlogn$
Solution-

We compare the given recurrence relation with $T(n) = aT(n/b) + \theta (nklogpn)$.

Then, we have-$a = 2, b = 2, k = 1, p = 1$
Now, $a = 2$ and $bk = 21 = 2$.
Clearly, $a = bk$.
So, we follow case-02.
Since $p = 1$, so we have-
$T(n) = \theta (nlogba.logp+1n)$
$T(n) = \theta (nlog22.log1+1n)$

Thus,

$T(n) = \theta (nlog2n)$

03:Solve the following recurrence relation using Master's theorem-

$T(n) = 2T(n/4) + n0.51$

Solution- We compare the given recurrence relation with $T(n) = aT(n/b) + \theta (nklogpn)$.
Then, we have-$a = 2, b = 4, k = 0.51, p = 0$
Now, $a = 2$ and $bk = 40.51 = 2.0279$.
Clearly, $a < bk$.
So, we follow case-03.
Since $p = 0$, so we have-

$T(n) = \theta (n\ k\ \log_p n)$

$T(n) = \theta (n0.51log0n)$

Thus,

$T(n) = \theta (n0.51)$

04: Solve the following recurrence relation using Master's theorem-

$T(n) = \sqrt{2}T(n/2) + logn$

Solution-
We compare the given recurrence relation with $T(n) = aT(n/b) + \theta (nklogpn)$.
Then, we have-$a = \sqrt{2}, b = 2, k = 0, p = 1$

Now, $a = \sqrt{2} = 1.414$ and $bk = 20 = 1$.

Clearly, $a > bk$.

So, we follow case-01.
So, we have-

$T(n) = \theta (n^{\log_b a})$

$T(n) = \theta (n^{\log_2 \sqrt{2}})$

$T(n) = \theta (n^{1/2})$
Thus,

$T(n) = \theta (\sqrt{n})$

Problem-05:

Solve the following recurrence relation using Master's theorem-

$T(n) = 8T(n/4) - n^2 \log n$
Solution-
The given recurrence relation does not correspond to the general form of Master's theorem.
So, it can not be solved using Master's theorem.

06:Solve the following recurrence relation using Master's theorem-

$T(n) = 3T(n/3) + n/2$
Solution-We write the given recurrence relation as $T(n) = 3T(n/3) + n$.
This is because in the general form, we have θ for function f(n) which hides constants in it.
Now, we can easily apply Master's theorem.

We compare the given recurrence relation with $T(n) = aT(n/b) + \theta (n^k \log^p n)$.

Then, we have-$a = 3, b = 3, k = 1, p = 0$

Now, $a = 3$ and $b^k = 3^1 = 3$.

Clearly, $a = b^k$.

So, we follow case-02.

Since $p = 0$, so we have-

$T(n) = \theta (n^{\log_b a} . \log^{p+1} n)$

$T(n) = \theta (n^{\log_3 3} . \log^{0+1} n)$

$T(n) = \theta (n^1 . \log^1 n)$

Thus,

$T(n) = \theta (n \log n)$

07:Form a recurrence relation for the following code and solve it using Master's theorem-

```
A(n)
{
  if(n<=1)
    return 1;
  else
    return A(√n);
}
```

Solution-
We write a recurrence relation for the given code as $T(n) = T(\sqrt{n}) + 1$.
Here 1 = Constant time taken for comparing and returning the value.
We can not directly apply Master's Theorem on this recurrence relation.
This is because it does not correspond to the general form of Master's theorem.
However, we can modify and bring it in the general form to apply Master's theorem.
Let-

$n = 2m$ ……(1)

Then-

$T(2m) = T(2m/2) + 1$
Now, let $T(2m) = S(m)$, then $T(2m/2) = S(m/2)$
So, we have-

$S(m) = S(m/2) + 1$
Now, we can easily apply Master's Theorem.
We compare the given recurrence relation with $S(m) = aS(m/b) + \theta (mklogpm)$.
Then, we have-
$a = 1, b = 2, k = 0, p = 0$
Now, $a = 1$ and $bk = 20 = 1$.

Clearly, $a = bk$.
So, we follow case-02.
Since $p = 0$, so we have-

$S(m) = \theta (mlogba.logp+1m)$

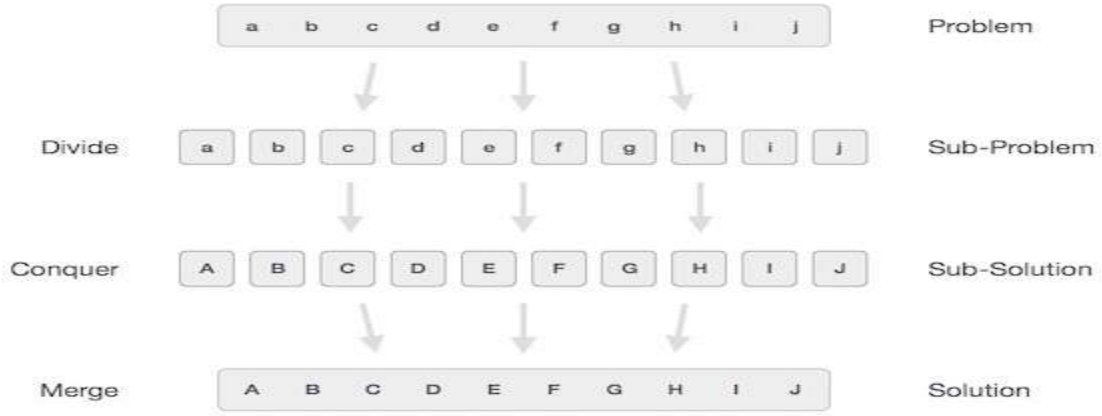
$S(m) = \theta (mlog21.log0+1m)$

$S(m) = \theta (m0.log1m)$
Thus,
$S(m) = \theta(logm)$ ……(2)
Now,
From (1), we have $n = 2m$.
So, $logn = mlog2$ which implies $m = log2n$.
Substituting in (2), we get-
$S(m) = \theta(loglog2n)$
Or

$T(n) = \theta (loglog2n)$

DIVIDE-AND-CONQUER

In divide and conquer approach, the problem in hand, is divided into smaller sub-problems and then each problem is solved independently. When we keep on dividing the sub problems into even smaller sub-problems, we may eventually reach a stage where no more division is possible. Those "atomic" smallest possible sub-problem (fractions) are solved. The solution of all sub-problems is finally merged in order to obtain the solution of an original problem.



Broadly, we can understand divide-and-conquer approach in a three-step process.

Divide/Break

This step involves breaking the problem into smaller sub-problems. Sub-problems should represent a part of the original problem. This step generally takes a recursive approach to divide the problem until no sub-problem is further divisible. At this stage, sub-problems become atomic in nature but still represent some part of the actual problem.

Conquer/Solve

This step receives a lot of smaller sub-problems to be solved. Generally, at this level, the problems are considered 'solved' on their own.

Merge/Combine

When the smaller sub-problems are solved, this stage recursively combines them until they formulate a solution of the original problem. This algorithmic approach works recursively and conquers & merges steps works so close that they appear as one.

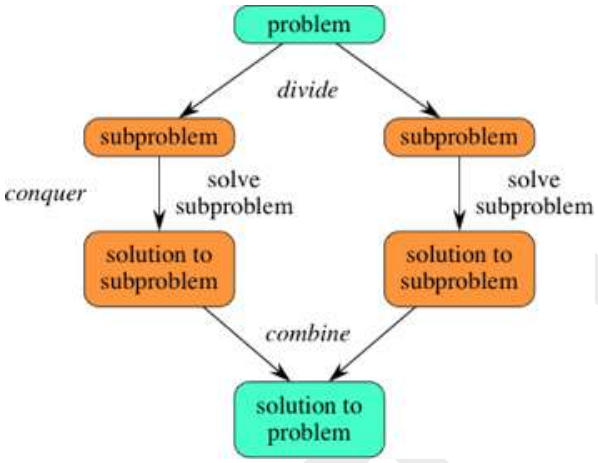
Examples

The following computer algorithms are based on divide-and-conquer programming approach

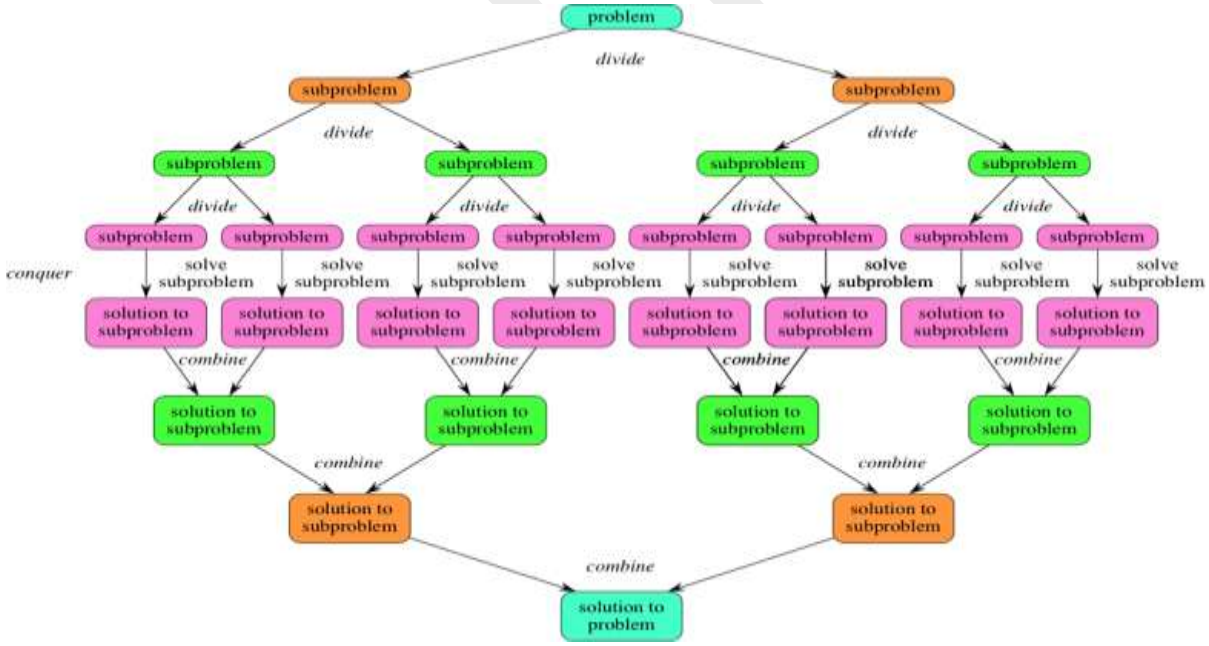
- Merge Sort
- Quick Sort
- Binary Search
- Strassen's Matrix Multiplication

There are various ways available to solve any computer problem, but the mentioned are a good example of divide and conquer approach.

You can easily remember the steps of a divide-and-conquer algorithm as *divide, conquer, combine*. Here's how to view one step, assuming that each divide step creates two sub problems (though some divide-and-conquer algorithms create more than two):



If we expand out two more recursive steps, it looks like this:



Advantages of Divide and Conquer

The most recognizable benefit of the divide and conquer paradigm is that it allows us to solve difficult problem, such as the Tower of Hanoi, which is a mathematical game or puzzle. Being given a difficult problem can often be discouraging if there is no idea how to go about solving it. However, with the divide and conquer method, it reduces the degree of difficulty since it divides the problem into sub problems that are easily solvable, and usually runs faster than other algorithms would.

It also uses memory caches effectively. When the sub problems become simple enough, they can be solved within a cache, without having to access the slower main memory.

Disadvantages of Divide and Conquer

One of the most common issues with this sort of algorithm is the fact that the recursion is slow, which in some cases outweighs any advantages of this divide and conquer process.

Sometimes it can become more complicated than a basic iterative approach, especially in cases with a large n.

In other words, if someone wanted to add a large amount of numbers together, if they just create a simple loop to add them together, it would turn out to be a much simpler approach than it would be to divide the numbers up into two groups, add these groups recursively, and then add the sums of the two groups together.

Time Complexity

The complexity of the divide and conquer algorithm is calculated using the master theorem.

$T(n) = a\ T(n/b) + f(n)$

where,

n = size of input

a = number of sub problems in the recursion

n/b = size of each sub problem. All sub problems are assumed to have the same size.

f(n) = cost of the work done outside the recursive call, which includes the cost of dividing the problem and cost of merging the solutions

Binary Search

1. In Binary Search technique, we search an element in a sorted array by recursively dividing the interval in half.

2. Firstly, we take the whole array as an interval.

3. If the Pivot Element (the item to be searched) is less than the item in the middle of the interval, We discard the second half of the list and recursively repeat the process for the first half of the list by calculating the new middle and last element.

4. If the Pivot Element (the item to be searched) is greater than the item in the middle of the interval, we discard the first half of the list and work recursively on the second half by calculating the new beginning and middle element.

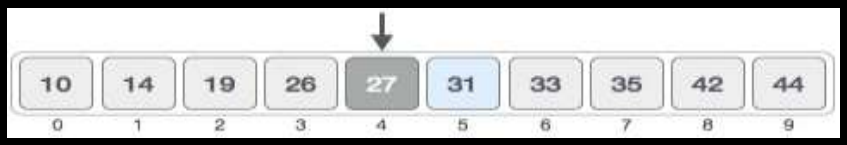
5. Repeatedly, check until the value is found or interval is empty.

Algorithm: Binary-Search (numbers [], x, l, r)

if l = r then
  return l
else
  m :=

First, we shall determine half of t
mid = low + (high -

Here it is, 0 + (9 - (



Now we comp
We find
than 27
the upp

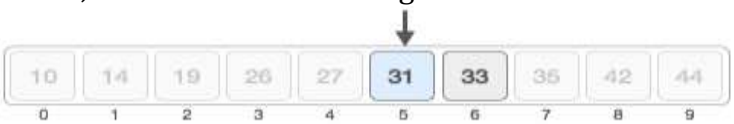
We char
low = mi
mid = lo

Our new

31.

The value stored at location 7 is not a match, rather it is more than what we are looking for. So, the value must be in the lower part from this location.

Hence, we calculate the mid again. This time it is 5.



We compare the value stored at location 5 with our target value. We find that it is a match.

We conclude that the target value 31 is stored at location 5.

Analysis:

1. **Input:** an array A of size n, already sorted in the ascending or descending order.
2. **Output:** analyze to search an element item in the sorted array of size n.
3. **Logic:** Let T (n) = number of comparisons of an item with n elements in a sorted array.

- Set BEG = 1 and END = n
- Find mid = $\text{int}\left(\frac{beg+end}{2}\right)$
- Compare the search item with the mid item.

Case 1: item = A[mid], then LOC = mid, but it the best case and T (n) = 1

Case 2: item ≠A [mid], then we will split the array into two equal parts of size $\frac{n}{2}$.
And again find the midpoint of the half-sorted array and compare with search element.
Repeat the same process until a search element is found.

T (n) = $T\left(\frac{n}{2}\right)+1$ ..... (Equation 1)

{Time to compare the search element with mid element, then with half of the selected half part of array}

At least there will be only one term left that's why that term will compare out, and only

$T\left(\frac{n}{2}\right) = T\left(\frac{n}{2^2}\right) + 1$, putting $\frac{n}{2}$ in place of n.

Then we get: $T(n) = \left(T\left(\frac{n}{2^2}\right) + 1\right) + 1$.....By putting $T\frac{n}{2}$ in (1) equation

$T(n) = T\left(\frac{n}{2^2}\right) + 2$..... (Equation 2)

$T\left(\frac{n}{2^2}\right) = T\left(\frac{n}{2^3}\right) + 1$..... Putting $\frac{n}{2}$ in place of n in eq 1.

$T(n) = T\left(\frac{n}{2^3}\right) + 1 + 2$

$T(n) = T\left(\frac{n}{2^3}\right) + 3$..... (Equation 3)

$T\left(\frac{n}{2^3}\right) = T\left(\frac{n}{2^4}\right) + 1$..... Putting $\frac{n}{3}$ in place of n in eq1

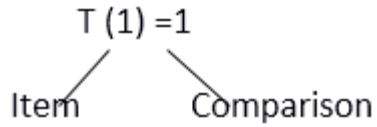
Put $T\left(\frac{n}{2^3}\right)$ in eq (3)

$T(n) = T\left(\frac{n}{2^4}\right) + 4$

Repeat the same process ith times

$T(n) = T\left(\frac{n}{2^i}\right) + i$ .....

Stopping Condition: T (1) =1



one comparison be done that's why

Is the last term of the equation and it will be equal to 1

$\frac{n}{2^i} = 1$

$n = 2^i$

Applying log both sides

$\log n = \log_2 i$

Log n = i log 2

$\frac{n}{2^i}$ **Is the last term of the equation and it will be equal to 1**

$\frac{\log n}{\log 2} = 1$

$\log_2 n = i$

$\frac{n}{2^i} = 1$ **as in eq 5**

$T(n) = T\left(\frac{n}{2^i}\right) + i$

$= T(1) + i$

$= 1 + i$ ..... T (1) =1 by stopping condition

$= 1 + \log_2 n$

$= \log_2 n$ ..... (1 is a constant that's why ignore it)

Therefore, binary search is of order o ($\log_2 n$)

Quick Sort

It is used on the principle of divide-and-conquer. Quick sort is an algorithm of choice in many situations as it is not difficult to implement. It is a good general purpose sort and it consumes relatively fewer resources during execution.

Quick sort works by partitioning a given array ***A[p ... r]*** into two non-empty sub array ***A[p ... q]*** and ***A[q+1 ... r]*** such that every key in ***A[p ... q]*** is less than or equal to every key in ***A[q+1 ... r]***.

Then, the two sub-arrays are sorted by recursive calls to Quick sort. The exact position of the partition depends on the given array and index ***q*** is computed as a part of the partitioning procedure.

Algorithm: Quick-Sort (A, p, r)

```
if p < r then
  qPartition (A, p, r)
  Quick-Sort (A, p, q)
  Quick-Sort (A, q + r, r)
```

Note that to sort the entire array, the initial call should be ***Quick-Sort (A, 1, length[A])***

As a first step, Quick Sort chooses one of the items in the array to be sorted as pivot. Then, the array is partitioned on either side of the pivot. Elements that are less than or equal to pivot will move towards the left, while the elements that are greater than or equal to pivot will move towards the right.

Partitioning the Array

Partitioning procedure rearranges the sub-arrays in-place.

Function: qPartition (A, p, r)

```
x ← A[p]
i ← p-1
j ← r+1
while TRUE do
  Repeat j ← j - 1
  until A[j] ≤ x
  Repeat i← i+1
  until A[i] ≥ x
  if i < j then
    exchange A[i] ↔ A[j]
  else
    return j
```

example

arr[] = {10, 80, 30, 90, 40, 50, 70}
Indexes: 0 1 2 3 4 5 6

low = 0, high = 6, pivot = arr[h] = 70
Initialize index of smaller element, **i = -1**

Traverse elements from j = low to high-1
j = 0 : Since arr[j] <= pivot, do i++ and swap(arr[i], arr[j])
i = 0
arr[] = {10, 80, 30, 90, 40, 50, 70} // No change as i and j are same

j = 1 : Since arr[j] > pivot, do nothing
// No change in i and arr[]

j = 2 : Since arr[j] <= pivot, do i++ and swap(arr[i], arr[j])
i = 1
arr[] = {10, 30, 80, 90, 40, 50, 70} // We swap 80 and 30

j = 3 : Since arr[j] > pivot, do nothing
// No change in i and arr[]

j = 4 : Since arr[j] <= pivot, do i++ and swap(arr[i], arr[j])
i = 2
arr[] = {10, 30, 40, 90, 80, 50, 70} // 80 and 40 Swapped
j = 5 : Since arr[j] <= pivot, do i++ and swap arr[i] with arr[j]
i = 3

arr[] = {10, 30, 40, 50, 80, 90, 70} // 90 and 50 Swapped

We come out of loop because j is now equal to high-1.
Finally we place pivot at correct position by swapping arr[i+1] and arr[high] (or pivot)
arr[] = {10, 30, 40, 50, 70, 90, 80} // 80 and 70 Swapped

Now 70 is at its correct place. All elements smaller than 70 are before it and all elements greater than 70 are after it.

Time complexity calculation

we can formulate a recurrence the exact excepted time as

$$T(N)=\sum_{p=1}^{n}\frac{1}{n}(T(P-1)+T(N-P))+N-1$$

Each possible Pivot P is selected with equal probability. The comparisons needed to do is (N-1)

Now

$$T(N)=\sum_{p=1}^{n}\frac{1}{n}(T(P-1)+T(N-P))+N-1$$

$$=\frac{2}{N}\sum_{p=1}^{n}(T(P-1)+N-1$$

Now multiply with 'n' on both sides, we get

$$N.T(N)=N.\frac{2}{N}\sum_{p=1}^{n}(T(P-1)+(N-1).N$$

$$2.\sum_{p=1}^{n}(T(P-1)+(N-1).N \quad \text{---------- Equations (1)}$$

Now consider N=N-1,then we get

$$(N-1).T(N-1)=2.\sum_{p=1}^{n-1}(T(P-1)+((N-1)-1).(N-1)$$

$$2.\sum_{p=1}^{n-1}(T(P-1))+(N-2).(N-1) \text{---------- Equations (2)}$$

Subtract Equations (2)- Equations (1) we get

$$(N-1).T(N-1) - N.T(N)= 2T(N-1)+2(N-1)$$

After rearranging we get

$$\frac{T(N)}{N+1}=\frac{T(N-1)}{N}+\frac{2(N-1)}{N(N+1)}$$

Now consider $A_N = \frac{T(N)}{N+1}$

$$A_N = A_{N-1}+\frac{2(N-1)}{N(N+1)}$$

$$A_N = \sum_{I=0}^{N}\frac{2(i-1)}{i(i+1)}$$

$$A_N = 2\sum_{i=0}^{N}\frac{1}{i+1}$$

As per Harmonic numbers Hn is $Hn=\sum_{i=1}^{N}\frac{1}{i}\approx \log n$ so $An \approx 2 \log n$

$T(N)/N+1\approx 2 \log N$

$T(N)=2(N+1) \log N$

Therefore the average time complexity is O(log n)

Analysis

Where as if partitioning leads to almost equal subarrays, then the running time is the best, with time complexity as O(n*log n).
Worst Case Time Complexity [Big- omega]: Ω((n2)
Best Case Time Complexity [Big-]: O(n*log n)
Space Complexity: O(n*log n)

Advantages

- It is in-place since it uses only a small auxiliary stack.
- It requires only n (log n) time to sort n items.
- It has an extremely short inner loop.
- This algorithm has been subjected to a thorough mathematical analysis, a very precise statement can be made about performance issues.

Disadvantages

- It is recursive. Especially, if recursion is not available, the implementation is extremely complicated.
- It requires quadratic (i.e., n2) time in the worst-case.
- It is fragile, i.e. a simple mistake in the implementation can go unnoticed and cause it to perform badly.

Merge Sort

Merge sort is one of the most efficient sorting algorithms. It works on the principle of Divide and Conquer. Merge sort repeatedly breaks down a list into several sublists until each sublist consists of a single element and merging those sublists in a manner that results into a sorted list.

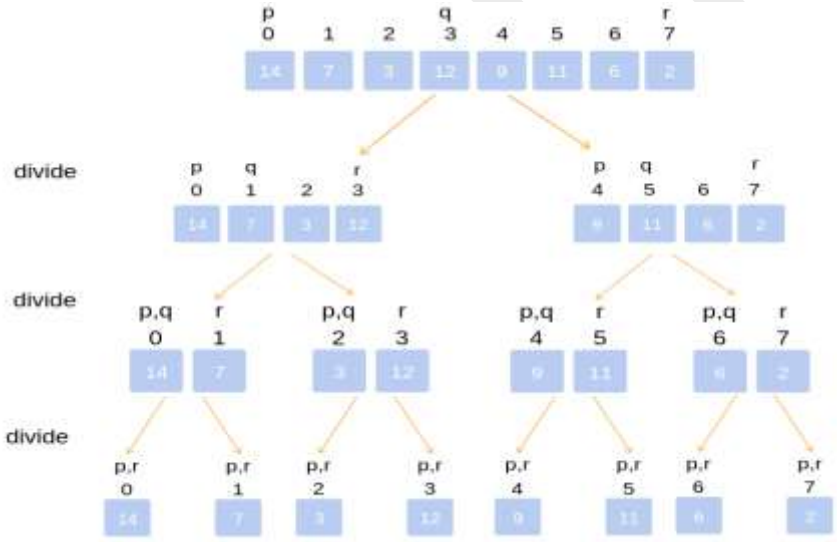
A merge sort works as follows:

Top-down Merge Sort Implementation:

The top-down merge sort approach is the methodology which uses recursion mechanism. It starts at the Top and proceeds downwards, with each recursive turn asking the same question such as "What is required to be done to sort the array?" and having the answer as, "split the array into two, make a recursive call, and merge the results.", until one gets to the bottom of the array-tree.

Example: Let us consider an example to understand the approach better.

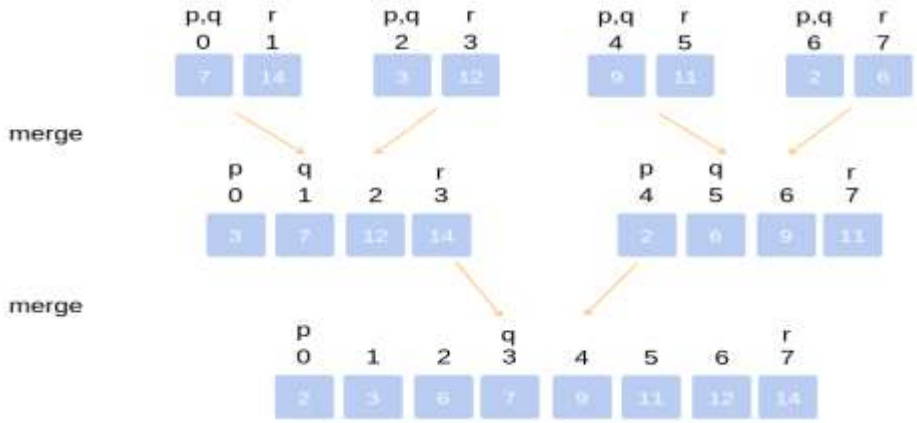
Divide the unsorted list into n sublists, each comprising 1 element (a list of 1 element is supposed sorted).



Repeatedly merge sublists to produce newly sorted sublists until there is only 1 sublist remaining. This will be the sorted list.

Merging of two lists done as follows:

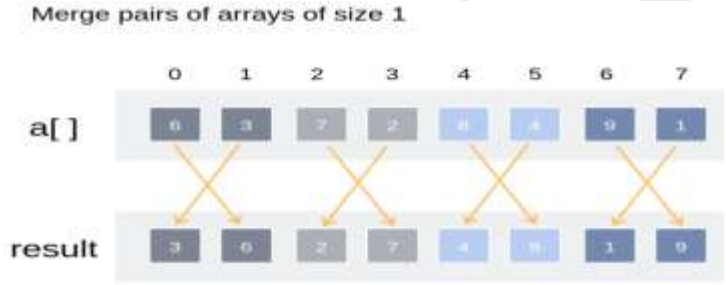
The first element of both lists is compared. If sorting in ascending order, the smaller element among two becomes a new element of the sorted list. This procedure is repeated until both the smaller sublists are empty and the newly combined sublist covers all the elements of both the sublists.



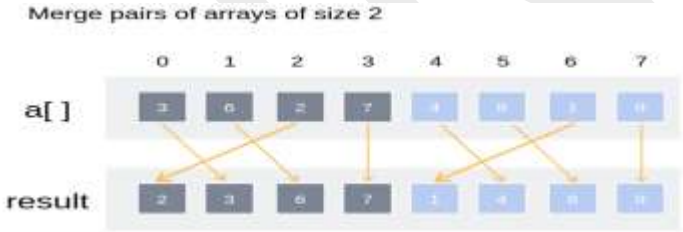
Bottom-Up Merge Sort Implementation:

The Bottom-Up merge sort approach uses iterative methodology. It starts with the "single-element" array, and combines two adjacent elements and also sorting the two at the same time. The combined-sorted arrays are again combined and sorted with each other until one single unit of sorted array is achieved.

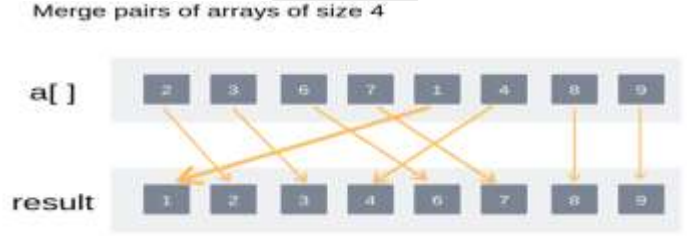
Example: Let us understand the concept with the following example.



Iteration (2)



Iteration (3)



ALGORITHM-MERGE SORT

If p<r

Then q → (p+ r)/2

MERGE-SORT (A, p, q)

MERGE-SORT (A, q+1,r)

MERGE (A, p, q, r)

FUNCTIONS: MERGE (A, p, q, r)
1.n 1 = q-p+1
2. n 2= r-q
3. create arrays [1.....n 1 + 1] and R [1.....n 2 +1]
4. for i ← 1 to n 1
5. do [i] ← A [p+ i-1]
6. for j ← 1 to n2
7. do R[j] ← A[q + j]
8. L [n 1+ 1] ← ∞
9. R[n 2+ 1] ← ∞
10. I ← 1
11. J ← 1
12. For k ← p to r
13. Do if L [i] ≤ R[j]
14. then A[k] ← L[i]
15. i ← i +1
16. else A[k] ← R[j]
17. j ← j+1

Analysis of Merge Sort:

Let T (n) be the total time taken by the Merge Sort algorithm.

Sorting two halves will take at the most 2Tn/2time.

When we merge the sorted lists, we come up with a total n-1 comparison because the last element which is left will need to be copied down in the combined list, and there will be no comparison.

Thus, the relational formula will be

$$T(n) = 2T\left(\frac{n}{2}\right) + n - 1$$

But we ignore '-1' because the element will take some time to be copied in merge lists.

So $T(n) = 2T\left(\frac{n}{2}\right) + n$...equation 1

Note: Stopping Condition T (1) =0 because at last, there will be only 1 element left that need to be copied, and there will be no comparison.

Putting $n=\frac{n}{2}$ in place of n in .....equation 1

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{2^2}\right) + \frac{n}{2}$$.....equation2

Put 2 equation in 1 equation

$T(n) = 2\left[2T\left(\frac{n}{2^2}\right) + \frac{n}{2}\right] + n$

$= 2^2T\left(\frac{n}{2^2}\right) + \frac{2n}{2} + n$

$T(n) = 2^2\,T\left(\frac{n}{2^2}\right) + 2n$.....equation 3

Putting $n = \frac{n}{2^2}$ in equation 1

$T\left(\frac{n}{2^2}\right) = 2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}$.....equation4

Putting 4 equation in 3 equation

$T(n) = 2^2\left[2T\left(\frac{n}{2^3}\right) + \frac{n}{2^2}\right] + 2n$

$T(n) = 2^3\,T\left(\frac{n}{2^3}\right) + n + 2n$

$T(n) = 2^3\,T\left(\frac{n}{2^3}\right) + 3n$.....equation5

From eq 1, eq3, eq 5.....we get

$T(n) = 2^i\,T\left(\frac{n}{2^i}\right) + in$.....equation6

From Stopping Condition:

$\frac{n}{2^i} = 1$ And $T\left(\frac{n}{2^i}\right)=0$

$n = 2^i$

Apply log both sides:

log n=log2i

logn= i log2

$\frac{\log n}{\log 2}$ =i

log2n=i

From 6 equation

$T(n) = 2^i\,T\left(\frac{n}{2^i}\right) + in$

$= 2^i \times 0 + \log_2 n \cdot n$

= T (n) = n.log n

Best Case Complexity: The merge sort algorithm has a best-case time complexity of O(n*log n) for the already sorted array.

Average Case Complexity: The average-case time complexity for the merge sort algorithm is O(n*log n), which happens when 2 or more elements are jumbled, i.e., neither in the ascending order nor in the descending order.

Worst Case Complexity: The worst-case time complexity is also O(n*log n), which occurs when we sort the descending order of an array into the ascending order.

Space Complexity: The space complexity of merge sort is O(n).

Merge Sort Applications

The concept of merge sort is applicable in the following areas:
Inversion count problem
External sorting
E-commerce applications

Stassen's Matrix Multiplication

Let us consider two matrices X and Y. We want to calculate the resultant matrix Z by multiplying X and Y.

Naïve Method

First, we will discuss naïve method and its complexity. Here, we are calculating Z = X × Y. Using Naïve method, two matrices (X and Y) can be multiplied if the order of these matrices are p × q and q × r. Following is the algorithm.

Algorithm: Matrix-Multiplication (X, Y, Z)

```
for i = 1 to p do
  for j = 1 to r do
    Z[i,j] := 0
    for k = 1 to q do
      Z[i,j] := Z[i,j] + X[i,k] × Y[k,j]
```

Strassen algorithm is a recursive method for matrix multiplication where we divide the matrix into 4 sub-matrices of dimensions n/2 x n/2 in each recursive step.
For example, consider two 4 x 4 matrices A and B that we need to multiply. A 4 x 4 can be divided into four 2 x 2 matrices.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

Here, A_{i} and B_{i} are 2 x 2 matrices.
Now, we can calculate the product of A and B (matrix C) with the following formulas:

$$P_1 = A_{11} \cdot (B_{12} - B_{22})$$
$$P_2 = (A_{11} + A_{12}) \cdot B_{22}$$
$$P_3 = (A_{21} + A_{22}) \cdot B_{11}$$
$$P_4 = A_{22} \cdot (B_{21} - B_{11})$$
$$P_5 = (A_{11} + A_{22}) \cdot (B_{11} + B_{22})$$
$$P_6 = (A_{12} - A_{22}) \cdot (B_{21} + B_{22})$$
$$P_7 = (A_{11} - A_{21}) \cdot (B_{11} + B_{12})$$

$$P_5 + P_4 - P_2 + P_6 = C_{11}$$
$$P_1 + P_2 = C_{12}$$
$$P_3 + P_4 = C_{21}$$
$$P_5 + P_1 - P_3 - P_7 = C_{22}$$

Complexity

Here, we assume that integer operations take O(1) time. There are three for loops in this algorithm and one is nested in other. Hence, the algorithm takes O(n3) time to execute.

This leads to a divide-and-conquer algorithm with running time T(n) = 7T(n/2) + _(n2)

– We only need to perform 7 multiplications recursively.
– Division/Combination can still be performed in _(n2) time.

$$
\begin{aligned}
T(n) &= 7T(n/2) + n^2 \\
&= n^2 + 7(7T(\frac{n}{2^2}) + (\frac{n}{2})^2) \\
&= n^2 + (\frac{7}{2^2})n^2 + 7^2T(\frac{n}{2^2}) \\
&= n^2 + (\frac{7}{2^2})n^2 + 7^2(7T(\frac{n}{2^3}) + (\frac{n}{2^2})^2) \\
&= n^2 + (\frac{7}{2^2})n^2 + (\frac{7}{2^2})^2 \cdot n^2 + 7^3T(\frac{n}{2^3}) \\
&= n^2 + (\frac{7}{2^2})n^2 + (\frac{7}{2^2})^2n^2 + (\frac{7}{2^2})^3n^2 .... + (\frac{7}{2^2})^{\log n-1}n^2 + 7^{\log n} \\
&= \sum_{i=0}^{\log n-1} (\frac{7}{2^2})^i n^2 + 7^{\log n} \\
&= n^2 \cdot \Theta((\frac{7}{2^2})^{\log n-1}) + 7^{\log n} \\
&= n^2 \cdot \Theta(\frac{7^{\log n}}{(2^2)^{\log n}}) + 7^{\log n} \\
&= n^2 \cdot \Theta(\frac{7^{\log n}}{n^2}) + 7^{\log n} \\
&= \Theta(7^{\log n})
\end{aligned}
$$

– Now we have the following:

$$
\begin{aligned}
7^{\log n} &= 7^{\frac{\log_7 n}{\log_7 2}} \\
&= (7^{\log_7 n})^{(1/\log_7 2)} \\
&= n^{(1/\log_7 2)} \\
&= n^{\frac{\log_2 7}{\log_2 2}} \\
&= n^{\log 7}
\end{aligned}
$$

– Or in general: $a^{\log_k n} = n^{\log_k a}$

So the solution is $T(n) = \Theta(n^{\log 7}) = \Theta(n^{2.81...})$

- Note:
  - We are 'hiding' a much bigger constant in $\Theta()$ than before.
  - Currently best known bound is $O(n^{2.376..})$ (another method).
  - Lower bound is (trivially) $\Omega(n^2)$.

Objective questions

1. Worst-case – The maximum number of steps taken on any instance of size a.
2. Best-case – The minimum number of steps taken on any instance of size a.
3. Average case – An average number of steps taken on any instance of size a.
4. Amortized – A sequence of operations applied to the input of size a averaged over time.
5. Time complexity: Time required to run an algorithm in terms of the size of the input.
6. Space complexity: amount of memory an algorithm takes in terms of the size of input to the algorithm.
7. Execution time of an algorithm depends on the instruction set, processor speed, disk I/O speed.
8. Big-Oh (O) notation gives an upper bound for a function f(n) to within a constant factor.
9. Little o notation is used to describe an upper bound that cannot be tight.
10. Omega notation represents the lower bound of the running time of an algorithm.
11. According to the master theorem T(n) = aT(n/b) + f(n)
12. In master theorem T(n) = aT(n/b) + f(n) where n = size of input, a = number of sub problems in the recursion, n/b = size of each sub problem.
13. The steps involved in dive and conquer are divide, *conquer, combine*.
14. The disadvantage of divide and conquer is the process of recursion is slow
15. The algorithms like merge sort ,quick sort and binary search are based on divide and conquer algorithm
16. The steps in the divide and conquer process that takes a recursive approach is said to be divide/break
17. In binary search the mid element =(low+mid)/2
18. The best time complexity of binary search is O(1)
19. The average time complexity of binary search is O(log n)
20. The worst time complexity of binary search is O(log n)
21. The average time complexity of quick sort is O(n log n)
22. The best time complexity of quick sort is O(n log n)
23. The worst time complexity of quick sort is O(n log n)
24. The average time complexity of merge sort is O(n log n)
25. The best time complexity of merge sort is O(n log n)
26. The worst time complexity of merge sort is O(n log n)
27. The matrix multiplications algorithm's time complexity is O(n^3),
28. The matrix multiplications algorithm's requires 7 matrix multiplications and 10 additions and subtractions
29. The Strassen's matrix multiplications algorithm's time complexity is O (n^2.80).
30. The Strassen's matrix multiplications algorithm's requires 7 matrix multiplications and 18 additions and subtractions.
31. In analysis of algorithm, approximate relationship between the size of the job and the amount of work required to do is expressed by using _____

(a) Central tendency (b) Differential equation (c) Order of execution (d) Order of magnitude
Ans :Order of execution

32. Worst case efficiency of binary search is
(a) log2 n + 1 (b) n (c) N2 (d) 2n (e) log n.
Ans : log2 n + 1
33. For analyzing an algorithm, which is better computing time?
(a)O (100 Log N) (b) O (N) (c)O (2N) (d) O (N logN) (e) O (N2).
Ans :O (100 Log N)
34. What do you call the selected keys in the quick sort method?
(a) Outer key (b)Inner Key (c) Partition key(d) Pivot key (e) Recombine key.
Ans :c
35. The time complexity of the normal quick sort, randomized quick sort algorithms in the worst case is
(a) O(n2), O(n log n) (b) O(n2), O(n2) (c) O(n log n), O(n2) (d) O(n log n), O(n log n)
Ans :O(n2), O(n2)
36. Let there be an array of length 'N', and the selection sort algorithm is used to sort it, how many times a swap function is called to complete the execution?
(a) N log N times (b) log N times (c) N2 times (d) N-1 times
Ans :N-1 times
37. The Sorting methodwhich is used for external sort is
(a) Bubble sort (b) Quick sort (c) Merge sort (d) Radix sort
Ans :Radix sort
38. The amount of memory needs to run to completion is known as_____
a. Space complexity c. Worst case
b. Time complexity d. Best case
Ans: Space complexity
39. The amount of time needs to run to completion is known as_____
a. Space complexity b. Time complexity c. Worst case d. Best case
Ans: Time complexity
40. _____ is the minimum number of steps that can executed for the given parameters
a. Average case b. Time complexity c. Worst case d. Best case
Ans: Best case
41. _____ is the maximum number of steps that can executed for the given parameters
a. Average case b. Time complexity c. Worst-case d. Best case
Ans:Worst case
42. _____ is the average number of steps that can executed for the given parameters
a. Average case b. Time complexity c. Worst case d. Best case
Ans: Average Case
43. Testing of a program consists of 2 phases which are _____and_____

a. Average case & Worst case b. Time complexity & Space complexity c. Validation and checking errors d. Debugging and profiling
Ans: Debugging and profiling

44. Worst case time complexity of binary search is _____
a. O(n) b. O(logn)c. Θ(nlogn) d. Θ(logn)
Ans: Θ(logn)

45. Best case time complexity of binary search is _____
a. O(n) b. O(logn) c. Θ(nlogn) d. Θ(logn)
Ans: Θ(logn)

46. Average case time complexity of binary search is _____
a. O(n) b. O(logn) c. Θ(nlogn) d. Θ(logn)
Ans: Θ(logn)

47. Merge sort invented by _____
a. CARHOARE b. JOHN VON NEUMANN c. HAMILTON d. STRASSEN
Ans : JOHN VON NEUMANN

48. Quick sort invented by _____
a. CARHOARE b. JOHN VON NEUMANN c. HAMILTON d. STRASSEN
Ans : CARHOARE

49. Worst case time complexity of Quick sort is _____
a. O(n2log7) b. O(n2) c. O(nlogn) d. O(logn)
Ans : O(n2)

50. Best case time complexity of Quick sort is _____
a. O(n2logn) b. O(logn) c. O(nlogn) d. O(logn2)
Ans : O(nlogn)

Previous Questions

1. Solve the following recurrence T(n)=4T(n/2)+n where n>=1 and is power of 2
2. Write the non-recursive algorithm for finding the Fibonacci sequence and define its time complexity.
3. What are the different mathematical notations used for algorithm analysis.
4. Describe the performance analysis of an algorithm in detail.
5. Define the time complexity.
6. Define time complexity. Describe different notations used to represent time complexities.
7. Define algorithm. Write about algorithm design techniques.
8. Define Space Complexity. Compute space complexity for an algorithm to find factorial of a given number.
9. Consider the following recurrence equation:

$$T(n) = \begin{cases} 1 \; if \; n = 1 \\ T(n-1) + n^n \; other \; wise \end{cases}$$

10. Define recurrence equation? Find the time complexity of merge sort from recurrence relation using substitution method.
11. Write the pseudo code for binary search and analyze the time complexity .
12. Explain the general method of divide and conquer with an example.
13. Explain the properties of algorithm with real time example.
14. Explain Recursive Binary search algorithm with suitable examples.
15. Distinguish between Merge sort and quick sort.
16. What is stable sorting method? Is Merge sort a stable sorting method? Justify your answer.
17. Explain partition exchange sort algorithm and trace this algorithm for n =8 elements: 24,12, 35, 23,45,34,20,48
18. Briefly explain merge sort algorithm with suitable example and derive its time complexity.
19. Explain divide and conquer in detail.
20. Show how quick sort sorts the following sequences of keys 65, 70, 75, 80, 85, 60, 55, 50, 45 solve the recurrence relation of quick sort using substitution method.
21. Explain how divide and conquer method is used to implement Merge sort technique with its Time complexity.
22. Write an algorithm for Quick sort.
23. Write Divide – And – Conquer recursive Quick sort algorithm and analyze the algorithm for average time complexity.
24. Trace the quick sort algorithm to sort the list C, O, L, L, E, G, E in alphabetical order
25. Describe Strassen's matrix multiplication. Perform the multiplication operation on following two matrices using Strasen's technique.

$$\begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix} X \begin{bmatrix} 2 & 7 \\ 8 & 3 \end{bmatrix}$$

26. Write an algorithm for stressan's matrix multiplication and analyze the complexity of your Algorithm